

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN
ĐỒ ÁN TRÒ CHƠI BẢNG QUA ĐƯỜNG

NHÓM 10 – 21CLC05

Đoàn Ngọc Mai	-	21127104
Lê Nguyễn Kiều Oanh	-	21127129
Nguyễn Đức Vĩnh Hoà	-	21127609
Lê Phước Quang Huy	-	21127616

Môn học: Phương pháp lập trình hướng đối tượng (OOP)

Giáo viên hướng dẫn: Thầy Trương Toàn Thịnh

MỤC LỤC

I. Một số chức năng chính	3
II.Các class được sử dụng trong đồ án.....	3
1. Một số hàm xử lý màn hình:.....	3
2. Class Pos (position):	3
3. Class ConstVar:	3
4. Class Player:	4
5. Class Enemy:	5
6. Class Bird, Bus, Car, Dog:	5
7. Class Level:	5
8. Class Lane:	6
9. Class Road:	7
10. Class Map :	8
11. Class Gameplay.....	12

Báo cáo đồ án game băng qua đường

I. Một số chức năng chính

- Người chơi sẽ dùng bàn phím để điều khiển nhân vật vượt các chướng ngại vật (Các enemy) để về đích.
- Vượt đến lane thứ 10 thì nhân vật sẽ được qua màn mới với level cao hơn, đến level 5 nhân vật tiếp tục vượt qua thì sẽ chiến thắng.
- Người chơi sẽ thua khi nhân vật bị va chạm với các Enemy.
- Có chức năng Save và Load game để chơi lại khi muốn.
- Có chức năng Pause bao gồm resume với exit cho người chơi.
- Bên phải màn hình sẽ có bảng hướng dẫn người chơi.
- Ở phần Settings tại Menu game cho phép người chơi điều chỉnh mức độ khó của game (easy hoặc hard), và có thể tắt phần âm thanh nếu thấy khó chịu (sound on hoặc off).
- Và còn một số phần xử lý về mặt hình ảnh khác...

II. Các class được sử dụng trong đồ án

1. Một số hàm xử lý màn hình:

- **gotoXY()**: thiết lập vị trí con trỏ cho màn hình, hàm này giúp ta có khả năng di chuyển đến bất kỳ vị trí trong màn hình console.
- **txtColor()**: dùng để thay đổi màu sắc theo ý muốn.
- **fixConsoleWindow()**: Cố định màn hình chính, để tránh người dùng thay đổi kích thước console gây ảnh hưởng các đối tượng làm khó khăn trong quá trình tính toán.
- **Nocursortype()**: dùng để ẩn con trỏ chuột trên màn hình console

2. Class Pos (position):

- **Properties**: x, y (tọa độ trong game)
- **Method**: ngoài các constructor và destructor thì còn có các phương thức như getX(), getY(), setPos(int x, int y).
- **Chức năng chính**: dùng để điều chỉnh hay lấy tọa độ phục vụ class Player và Enemy.

3. Class ConstVar:

- Ở class này chỉ có thuộc tính kiểu bool ở tầm vực **public** theo dạng **static** là **isMute** và **isHard**.
- Mục đích chính là để điều chỉnh phần mức độ khó dễ của trò chơi và điều chỉnh phần âm thanh

Dòng	
1	<code>class constVar {</code>
2	<code>public:</code>
3	<code>static bool isMute;</code>

4	<code>static bool isHard; };</code>
---	-------------------------------------

4. Class Player:

- **Properties:**

- Class Pos.
- 2 mảng char 2 chiều: 1 dùng để chứa các ký tự vẽ nên nhân vật, 2 để chứa các khoảng trắng dùng để xóa nhân vật.
- 2 thuộc tính kiểu int mô tả chiều dài và rộng của nhân vật (width, height).
- 1 thuộc tính kiểu bool để kiểm tra nhân vật chết hay chưa.

- **Method:**

- **Constructor():** ở phương thức này chúng ta cấp phát động cho 2 mảng ký tự 2 chiều và bắt đầu lưu các ký tự để sử dụng. Và set vị trí đứng cho nhân vật, để cho biến kiểm tra nhân vật chết hay chưa là false.
- **Constructor(Pos):** tương tự như trên, chỉ thêm phần setPos tại tham số.
- Một số setter và getter như getX(), getY(), getWidth(), getHeight(), getPos(): tất cả đều thuộc nhân vật.
- **Move(Up, Down, Left, Right):** Các phương thức này giúp kiểm tra vị trí nhân vật còn trong map hay không và sẽ update vị trí lại cho nhân vật.

Dòng	
1	<code>void Player::MoveUp() {</code>
2	<code>if (pos.getX() <= Up_Map)</code>
3	<code>return;</code>
4	<code>pos.setPos(pos.getX() - 3, pos.getY()); }</code>

- 2 phương thức DeadPlayer(), checkDead(): bool dùng để set thuộc tính isDead = true và return isDead;
- **bool Collision(Pos, int, int):** 3 tham số truyền vào lần lượt là position của enemy bất kì, 2 tham số int là width và height của enemy đó. Phương thức sẽ kiểm tra xem Enemy đó có va chạm với nhân vật hay không, vì Position của enemy khi khởi tạo sẽ nằm ở phía trên bên trái nên khi xử lý chúng ta sẽ chia ra 2 phần đó là kiểm tra bên trái và kiểm tra bên phải

Dòng	
1	<code>bool Player::Collision(Pos pos, int w, int h) {</code>
2	<code>if (w == 5) {</code>
3	<code>if (this->getX() == pos.getX()) {</code>

4	<code>if (this->getY() <= pos.getY() && (pos.getY() - this->getY() <= 2)) // xử lý bên trái</code>
5	<code>return true;</code>
6	<code>if (this->getY() >= pos.getY() && getY() - pos.getY() <= 3) // xử lý bên phải</code>
7	<code>return true; }}</code>

5. Class Enemy:

- Đây là lớp cha của những Enemy như Bird, Car, Bus, Dog và 4 lớp trên đều có những thuộc tính và phương thức trùng nhau nên các phương thức thuộc class Enemy đã phân là phương thức ảo.
- **Properties:**
 - Class Pos.
 - 1 thuộc tính kiểu bool checkOut để kiểm tra Enemy đã ra khỏi map hay chưa.
- **Method:**
 - Ngoài những constructor và destructor thì có những setter và getter về tọa độ và chiều cao của Enemy
 - Void goOutMap(), bool isOutOfMap() dùng để set thuộc tính checkOut = true, và return checkOut.
 - Một số phương thức **virtual** như char** shape(), int getWidth(), int getType, void sound(). ~Enemy()

6. Class Bird, Bus, Car, Dog:

- Đây là những class kế thừa class Enemy
- **Properties:** chỉ có 1 mảng ký tự 2 chiều dùng để lưu những ký tự để vẽ nên hình ảnh enemy
- **Method:**
 - **Constructor** dùng để cấp phát động mảng ký tự 2 chiều và lưu những ký tự để vẽ.
 - **Destructor** để xóa đi bộ nhớ đã cấp phát động.
 - **Char** shape()** trả về mảng ký tự 2 chiều.
 - **getType()** trả về kiểu của Enemy đó (Bird = 0, Car = 1, Bus = 2, Dog = 3)
 - **sound()** để phát ra âm thanh khi va chạm theo enemy đó.

7. Class Level:

- **Properties:** Một số thuộc tính như level, và tốc độ của game.
- **Method:**

- Các setters, getters của các thuộc tính
- **Enemy* randNewEnemy(Pos):** phương thức này dùng để random ra 1 trong 4 enemy nhờ vào type ban đầu tại vị trí pos.
- **Enemy* getNewEnemy(Pos, int type):** phương thức này tương tự như trên nhưng không phải random mà là nhờ vào tham số type được truyền vào.
- **decNEnemy(int n):** dùng để trừ đi n enemy ban đầu.

8. Class Lane:

- **Properties:**

- **Vector<Enemy*>**
- Redlight, direction(hướng), speed, currentRow: Những thứ phải có trên 1 làn đường.

- **Method:**

- Constructor 4 tham số trên.
- Getter của 5 tham số trên.
- **ReverseRedLight():** dùng để đảo biến đèn xanh đỏ
- **Bool printNewEnemy(Pos, char**, int w, int h):** dùng để in ra hình 1 trong 4 enemy tại vị trí pos, trả về true nếu in được, trả về false nếu bị out map (left hoặc right map).
- **Void deleteOldEnemy(Pos, int w, int h):** dùng để xóa các enemy tại vị trí pos bằng cách in ra các khoảng trắng.
- **Int StepNewState(int t):** đây là phương thức xử lý đèn xanh đèn đỏ, hướng di chuyển, tốc độ di chuyển của các Enemy. Và phương thức này còn xử lý việc di chuyển của các Enemy trong 1 lane sang trái hoặc phải 1 đơn vị bằng cách xóa các Enemy cũ bằng phương thức deleteOldEnemy() và update vị trí mới bằng phương thức UpdatePosition() tại class Enemy. Sau đó in ra Enemy mới tại vị trí mới, nếu không in được thì Enemy đó đã ra khỏi map và chúng ta sẽ delete Enemy đó đi, Phương thức này trả về số Enemy bị xóa. Biến t ở đây được tăng dần sau mỗi vòng while.

if (((t % speed) != 0) && t != 0)
return nDelete;
....
// phần update vị trí cho các enemy

- Đây là phần xử lý về tốc độ, speed là thuộc tính mà ta sẽ có được sau khi tính từ class level. Vì t sẽ tăng lên 1 sau mỗi vòng while nên với speed càng bé thì tốc độ mỗi lần enemy update vị trí sẽ nhanh hơn nên nó di chuyển nhanh hơn và ngược lại.

Dòng	
1	vector <Enemy*> newEnemy;
2	newEnemy.reserve(100);

3	<code>for (int i = 0; i < (int)enemy.size(); ++i) {</code>
4	<code>Enemy* curEnemy = enemy[i];</code>
5	<code>int dy = -1;</code>
6	<code>if (direction)</code>
7	<code>dy = 1;</code>
8	<code>deleteOldEnemy(curEnemy->getPos(), curEnemy->getWidth(), curEnemy->getHeight());</code>
9	<code>curEnemy->updatePosition(0, dy);</code>
10	<code>bool canPrint = printNewEnemy(curEnemy->getPos(), curEnemy->shape(), curEnemy->getWidth(), curEnemy->getHeight());</code>
11	<code>if (!canPrint) {</code>
12	<code>curEnemy->goOutMap();</code>
13	<code>++nDelete; }</code>
14	<code>if (curEnemy->isOutOfMap())</code>
15	<code>delete curEnemy;</code>
16	<code>else</code>
17	<code>newEnemy.push_back(curEnemy);}</code>
18	<code>enemy = newEnemy;</code>
19	<code>return nDelete;</code>

- Phần này update vị trí và in ra các enemy trong 1 hàng như đã giải thích ở trên và giá trị trả về là số enemy đi ra khỏi map.

9. Class Road:

- **Properties:** 1 `vector<Lane*> subLanes` (có nghĩa là map sẽ gồm nhiều lane).
- **Method:**
 - Constructor sẽ cấp cho vector 1 dung lượng vừa đủ.
 - **Bool pushEnemy(int, Enemy*):** dùng để push enemy vào 1 lane nào đó trong subLanes.
 - **Void pushRoad(Lane*):** push 1 lane vào trong subLanes.
 - **Vector<Enemy*> listEnemy():** phương thức này dùng để lấy ra tất cả enemy trong map.

Dòng	
1	<code>vector<Enemy*> Road::listEnemy() {</code>
2	<code>vector<Enemy*> listEnemy, temp;</code>
3	<code>for (int i = 0; i < subLanes.size(); i++) {</code>
4	<code>temp = subLanes[i]->getEnemy();</code>
5	<code>listEnemy.insert(listEnemy.end(), temp.begin(), temp.end()); }</code>
6	<code>return listEnemy; }</code>

- 1 getter lấy ra thuộc tính subLanes.

- **Int StepNewState(int t):** đây là phương thức sử dụng phương thức StepNewState của class Lane, với ý nghĩa tương tự và áp dụng vào cho mọi lane thuộc map. Giá trị trả về là số enemy ra khỏi map của toàn bộ lane.

10. Class Map :

- **Properties:**

- Int width, height: chiều cao và rộng của Map.
- **Class Player: player**
- **Class Road: roadData**
- **Class Level: level**
- Int t = 0.
- Bool checkPause: để kiểm tra có dừng lại hay không.
- Method:
- ở Phần này những phần vẽ dùng nhiều những hàm như **gotoXY**, **txtColor**, và những ký tự đặc biệt trong mã ASCII. Và dùng vòng for lặp đi lặp lại để vẽ.
- Các phương thức như **printLosegame()**, **printLevelUp()**, **printCongrats()**: đều sử dụng 1 số kỹ thuật như trên để vẽ. Và ở phương thức **printLevelUp()** có dùng thêm kỹ thuật char **_getch()** để điều khiển bàn phím chọn Yes hoặc No.
- **Void drawMap():** phương thức này dùng để vẽ khung viền của map, phần viền board game và cả phần hướng dẫn trong board.
- **Int draw(Pos, char**, int w, int h):** phương thức này dùng để in ra mảng char 2 chiều tại vị trí pos dựa trên width và height.
- **Void deletePlayer():** dùng phương thức draw in ra những khoảng trống tại vị trí cũ của player, mục đích để xóa đi đối tượng cũ update và in ra vị trí mới.
- **Void updatePositionPlayer (char c):** phương thức này nhận tham số kiểu char bao gồm các ký tự (W, S, A, D) để update vị trí player, dùng các phương thức của class Player như MoveUp, MoveDown,... để update. Ở đầu phương thức này chúng ta sẽ gọi phương thức deletePlayer để xóa đi player ở vị trí cũ.

Dòng	
1	<code>void Map::UpdatePositionPlayer(char c) {</code>
2	<code>deletePlayer();</code>
3	<code>if (c == 'w')</code>
4	<code>player.MoveUp();</code>
5	<code>else if (c == 's')</code>
6	<code>player.MoveDown();</code>
7	<code>else if (c == 'a')</code>
8	<code>player.MoveLeft();</code>
9	<code>else if (c == 'd')</code>

10	player.MoveRight();
11	else
12	return; }

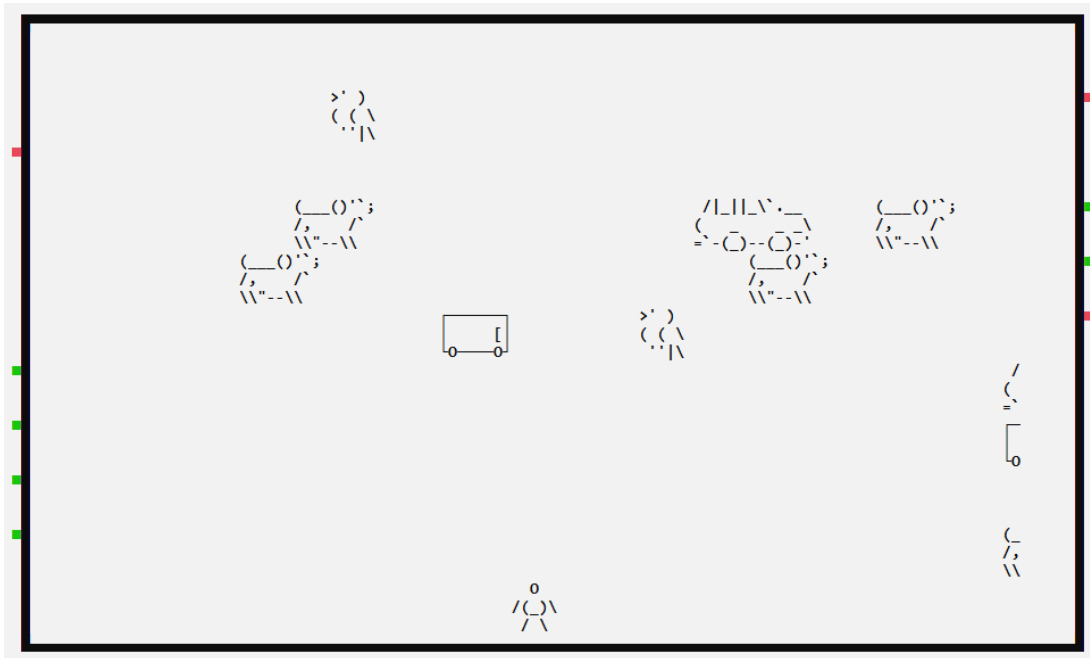
- **Void initializeNewState():** phương thức này dùng để khởi tạo random enemy và vị trí của enemy vào bất kỳ hàng nào với số lượng vừa đủ.

1	for (int i = 0; i < 10; ++i) {
2	padding[i] = 0;
3	int speed = rand() % (level.getMinSpeed() - level.getMaxSpeed() + 1) + level.getMaxSpeed();
4	bool direction = rand() % 2;
5	bool redLight = rand() % 2;
6	roadData.pushRow(new Lane(direction, speed, redLight, (i * 3) + 1)); }

- ở đây chúng ta sẽ dùng rand() để random các thuộc tính của 1 lane như speed, direction, redLight. Sau đó chúng ta sẽ push lane đó vào class Road, lần lượt như vậy chúng ta sẽ có được các lane.

1	Enemy* newEnemy;
2	Pos pos;
3	int tryCount = 10000;
4	while (tryCount--) {
5	int rRow = (rand() % 9) + 1;
6	padding[rRow] += (rand() % 20) + 9;
7	pos = Pos((rRow * 3) + 2, padding[rRow]);
8	newEnemy = level.RandomNewEnemy(pos);
9	if (!newEnemy)
10	break;
11	if (!roadData.pushEnemy(rRow, newEnemy)) {
12	level.decNEnemy(1);
13	delete newEnemy; }}

- Ở phần này chúng ta sẽ random và tạo ra những enemy trên các lane. Đầu tiên với rRow đó sẽ là số làn đường chúng ta cần pushEnemy (từ 1 đến 9). Padding[rRow] random mục đích để tạo khoảng cách giữa 2 enemy trên 1 lane. Pos ở đây với tọa độ x là trục dọc, y là trục ngang. rRow * 3 có nghĩa là 1 lane có chiều cao là 3 vì chiều mỗi enemy cũng là 3 (+ 2 ở đây để giúp enemy được in ra đúng tọa độ mình mong muốn). Padding[rRow] là tọa độ trục ngang để làm khoảng cách giữa các Enemy. Biến newEnemy được gán bằng phương thức **RandomNewEnemy** như đã giải thích ở trên. Nó sẽ trả về giá trị NULL nếu số enemy được tạo ra vượt quá số lượng đề ra ban đầu. Sau khi thực hiện xong phần trên nó sẽ được tương tự như sau:



- **void isDead():** phương thức này dùng để kiểm tra tất cả Enemy có va chạm vào player hay không.

1	<code>void Map::isDead() {</code>
2	<code>vector<Enemy*> enemyList = roadData.listEnemy();</code>
3	<code>for (int i = 0; i < enemyList.size(); i++) {</code>
4	<code>if (player.Collision(enemyList[i]->getPos(), enemyList[i]->getWidth() - 3, enemyList[i]->getHeight())) {</code>
5	<code>player.DeadPlayer();</code>
6	<code>deletePlayer();</code>
7	<code>. // phần xử lý sau khi va chạm như in logo lose game và các xử lý tương tự khác về mặt âm thanh</code>
8	<code>Return; }</code>
9	<code>}</code>

- **void randomNewState():** phương thức này sẽ sử dụng 1 vòng while(tryCount--) như phương thức **initializeNewState()** nhưng khác ở chỗ là pos của các enemy ở tọa độ trục ngang là 4 để khởi tạo vị trí enemy ở đầu hàng. Để trường hợp mỗi khi các enemy đi ra khỏi map thì sẽ có những enemy khác được khởi tạo vào.

1	<code>void Map::randomNewState() {</code>
2	<code> srand(time(NULL));</code>
	<code> . . . // vòng while(tryCount--)</code>
17	<code> ++t;</code>
18	<code> int tmp = roadData.StepNewState(t);</code>
19	<code> level.decNEnemy(tmp);</code>
20	<code> isDead();</code>

- Từ dòng 17 chúng ta sẽ thấy có biến **t** được tăng dần. Ở dòng 18 sẽ là phương thức thuộc class Road đã được giải thích ở trên. Nó sẽ giúp cho các enemy di chuyển sang trái hoặc phải thêm 1 đơn vị nếu **t % speed == 0** (vì phương thức **randomNewState** này sẽ nằm trong 1 vòng while và t được tăng liên tục). Giá trị trả ra đó là số enemy đi out map và sẽ bị trừ ra khỏi số lượng enemy. Ở dòng 20 là sẽ kiểm tra va chạm như đã giải thích ở trên.

- **Void WriteFileBin (int p, ofstream&):** phương thức này dùng để ghi dữ liệu ra file nhị phân.

1	<code>void Map::WriteFileBin(int p, ofstream& fileout) {</code>
2	<code> fileout.write((char*)&p, sizeof(int)); }</code>

- **Int ReadFileBin (ifstream&):** dùng để đọc dữ liệu ra từ file nhị phân.

1	<code>int Map::ReadFileBin(ifstream& filein) {</code>
2	<code> int p;</code>
3	<code> filein.read((char*)&p, sizeof(int));</code>
4	<code> return p;</code>

5	}
---	---

- **Void saveGame(string file):** phương thức này sử dụng phương thức **WriteFileBin** để ghi dữ liệu ra file nhị phân để lưu game. Những thuộc tính cần lưu như: level, tọa độ của player; tọa độ, hướng, speed, redLight của lane; số Enemy của map, và từng tọa độ của các enemy và kiểu của Enemy đó (dùng để phân loại 4 enemy).

1	<code>ofstream fileout("./LoadGame/" + file + ".bin", ios::out ios::binary);</code>
2	<code>WriteFileBin(level.getLevel(), fileout);</code>
3	<code>WriteFileBin(player.getX(), fileout);</code>
4	<code>WriteFileBin(player.getY(), fileout);</code>
	<code>. . . // các thông tin khác tương tự</code>

- **Void loadGame(string file):** sử dụng phương thức **ReadFileBin** để đọc dữ liệu ra từ file nhị phân, dựa trên thứ tự chúng ta đã lưu từ phương thức **saveGame**.

1	<code>ifstream filein("./LoadGame/" + file, ios::in ios::binary);</code>
2	<code>int lv = ReadFileBin(filein);</code>
3	<code>pX = ReadFileBin(filein);</code>
4	<code>pY = ReadFileBin(filein);</code>

- **Bool isWin():** kiểm tra đã win hay chưa dựa vào tọa độ trục dọc của player.
- **Bool isEnd():** trả về thuộc tính **checkDead**.

11. Class Gameplay

- **Properties:** class **Map**, bool **isLoad**, bool **isPause**.
- **Method:**
 - 1 số phương thức như **logoSaveGame()**, **logoCrossingRoad()**, **loadingBar()** dùng 1 số hàm cơ bản như ban đầu là **gotoXY**, **txtColor** để vẽ và **Sleep** để sự xuất hiện từng hình ảnh hợp lý hơn.

- **Void drawMenu():** ở phần này chúng ta dùng 2 vòng while. Ở vòng while ngoài sẽ chạy âm thanh, in **logoCrossingRoad**, vẽ viền cho menu và in ra những phần hiển thị như **New Game...** Tiếp theo là sẽ thêm 1 vòng while(), vòng lặp này dùng để điều khiển bàn phím chọn vào phần menu mình muốn, sử dụng kĩ thuật **char _getch()**

```

41 while (true)
42 {
43     char choice = _getch();
44     txtColor(0);
45     gotoXY(x + 4, y + 1);
46     cout << " NEW GAME ";
47     gotoXY(x + 4, y + 3);
48     cout << " LOAD GAME ";
49     gotoXY(x + 4, y + 5);
50     cout << " SETTINGS ";
51     gotoXY(x + 6, y + 7);
52     cout << " QUIT ";
53     if (choice == KEY_DOWN || choice == 'S' || choice == 's')
54     {
55         cpt++;
56         if (cpt > 4)
57             cpt = 1;
58     }
59     if (choice == KEY_UP || choice == 'W' || choice == 'w')
60     {
61         cpt--;
62         if (cpt < 1)
63             cpt = 4;
64     }
65     if (cpt == 1)
66     {
67         txtColor(155);
68         gotoXY(x + 4, y + 1);
69         cout << " NEW GAME ";
70         if (choice == KEY_ENTER)
71         {
72             loadingBar();
73             newGame();
74             break;
75         }
76     }
77     if (cpt == 2)
78     {

```

- Sau mỗi lần nhấn phím nó sẽ tăng hoặc giảm biến tạm, biến tạm bằng với số tương ứng nó sẽ đến phần đó và hiển thị phần đó lên, nếu tiếp tục nhấn Enter chúng ta sẽ vào thực hiện phương thức tương ứng.
 - **Bool newGame():** đây là phương thức dùng để chơi game và xử lý các tình huống trong màn hình gam.

1	<code>bool Gameplay::newGame() {</code>
2	<code> char key;</code>
3	<code> if (!isLoad) {</code>

4	map.~Map();
5	new(&map) Map(); }
6	...
7	map.drawMap();
8	isPause = false;
9	if (!isLoad)
10	map.initializeNewState();
11	isLoad = false;

- Phần này dùng để khởi tạo map, vẽ viền map và board map và khởi tạo những Enemy lên các lane.

1	while (!map.isEnd()) {
2	map.randomNewState();
3	if (isPause == true){
	...
	// dùng các kỹ thuật như phần menu để xử lý pause game
67	if (_kbhit()) {
68	char key = _getch();
69	if (key == 'W' key == 'w')
70	map.UpdatePositionPlayer('w');
	.
	.
	.
	// tương tự cho các phím khác
80	if (key == 'p' key == 'P')
81	isPause = true;
	.
	.
	.
	// tương tự cho các phím khác để save và load game
97	map.drawPlayer();}
99	if (map.isWin()) {
	// xử lý âm thanh
101	if (map.printLevelUp()) {
	.
	.
	.

```
// xử lý những phần về hình ảnh khi win game hoặc qua các màn tiếp theo
```

- Ở phần này có 1 vòng while để kiểm tra xem nếu player chết thì sẽ dừng vòng lặp, và phương thức `map.randomNewState()` sẽ tiếp tục di chuyển và khởi tạo các enemy mới sau mỗi vòng lặp tùy theo biến `t` (đã giải thích ở phần class `map`). **Hàm `kbhit()`**, đây là 1 hàm nhận tín hiệu từ bàn phím, nếu không có bất kì tín hiệu nào nó sẽ tiếp tục vòng while. Và cứ tiếp tục như vậy những enemy sẽ tiếp tục di chuyển, nếu nhận tín hiệu từ phím nào thì sẽ thực hiện các thao tác đó và vòng while vẫn cứ tiếp tục. Sau mỗi lần nhận các tín hiệu thì sẽ vẽ lại player bằng phương thức `map.drawPlayer()`, đây là cách để làm cho nhân vật di chuyển. Tại dòng 99, sẽ kiểm tra là đã win hay chưa (tọa độ nhân vật = tọa độ 2 của trục dọc), ở dòng 101 sẽ kiểm tra xem nhân vật đã chơi qua level 5 hay chưa, nếu chưa thì sẽ tiếp tục qua màn mới, còn nếu rồi thì người chơi đã win game này.

- **Void SaveGame():** phương thức này sử dụng phương thức `saveGame` thuộc class `Map` để lưu game vào file nhị phân, và ở phương này sẽ xử lý về mặt hình ảnh như `logoSaveGame`, hay là kỹ thuật nhập và sử dụng phím `BackSpace` để xóa ký tự.

1	<code>char key;</code>
2	<code>while ((key = _getch()) != 27) {</code>
3	<code>switch (key) {</code>
4	<code>case '\b':</code>
5	<code>if (filename.size() != 0) {</code>
6	<code>filename.pop_back();</code>
7	<code>gotoXY(50, 15);</code>
8	<code>cout << " ";</code>
9	<code>gotoXY(50, 15);</code>
10	<code>cout << filename; }</code>
11	<code>break;</code>
12	<code>case 13:</code>
13	<code>map.saveGame(filename);</code>
14	<code>break;</code>
15	<code>default:</code>
16	<code>filename.push_back(key);</code>
17	<code>gotoXY(50, 15);</code>
18	<code>cout << filename; }</code>
19	<code>if (key == 13) break;</code>

- Ở phần này tiếp tục sử dụng lệnh bắt phím nhấn `_getch()`, ta có 1 vòng lặp sẽ dừng lại cho đến khi nhấn phím `Esc` (tương đương mã `27`). Ở đây chúng ta sẽ có 3 case, đầu tiên là trường hợp nhấn phím `BackSpace` (`'\b'`), 2 là phím `enter`, và 3 là bất kỳ phím nào. Kỹ thuật sử dụng để thực hiện việc xóa khi nhấn `BackSpace` đó là sẽ `pop` 1 ký tự cuối cùng ra khỏi chuỗi ban đầu, sau đó in ra 1 chuỗi các khoảng trắng để xóa những ký tự đã in ra từ đầu, tiếp tục in ra lại chuỗi đó và chúng ta sẽ được chuỗi sẽ mất đi 1 ký tự ở cuối. Ở case 3, khi

nhấn bất kỳ phím nào khác thì chuỗi ban đầu sẽ push_back ký tự đó vào và chỉ việc ra lại vị trí đó thì sẽ có thêm 1 ký tự ở cuối dòng. Case 2, nhấn phím enter chúng ta sẽ gọi phương thức saveGame để lưu vào file nhị phân (như đã giải thích ở class Map).

- **Bool LoadGame():** đây là phương thức sử dụng phương thức loadGame() của class Map để lấy dữ liệu đã lưu. Và bổ sung thêm về phần xử lý hình ảnh.

- Chúng ta sẽ có 1 phương thức **getAllFileName(string)**, để lấy tất cả tên của các file đã lưu trong 1 folder nào đó và trả về 1 vector<string> (folder tên LoadGame).

```
vector<string> files = getAllFileName("LoadGame");
```

- Tiếp theo sẽ là phần xử lý về mặt hình ảnh và bắt tín hiệu bàn phím, sử dụng những kỹ thuật như gotoXY, getch(), _kbhit() và 1 số vòng lặp để có thể chọn tên file mình mong muốn. Và chỉ việc gọi phương loadGame(string) thuộc class Map với tham số là tên file thì sẽ lấy được dữ liệu đã lưu.

1	<code>while (true) {</code>
	<code>if (_kbhit()) {</code>
	<code>char key = _getch();</code>
	<code>if (key == 'w') { // xử lý hình ảnh }</code>
	<code>. //tương tự</code>
	<code>if (key == 13) {</code>
	<code>isLoad = true;</code>
	<code>map.loadGame(files[curPos]);</code>
	<code>. //1 vài xử lý khác Return true; }</code>

- **Void SettingsGame():** phương thức này sử dụng lại 2 thuộc tính dạng tĩnh thuộc tầm vực public của class ConstVar để tạo độ khó cho trò chơi và bật tắt âm thanh. Ngoài ra còn có 1 số xử lý khác về mặt hình ảnh.

	<code>.</code>
1	<code>if (cnt == 1) {</code>
2	<code>txtColor(155);</code>
3	<code>gotoXY(x + 6, y + 1); cout << "MODE:";</code>
4	<code>if (choice == KEY_ENTER) {</code>

5	<code>if (constVar::isHard == true)</code>
6	<code>constVar::isHard = false;</code>
7	<code>else</code>
8	<code>constVar::isHard = true;</code>
	· · · //Xử lý tương tự cho phần âm thanh.